

Module 32: Templates Meet OOP

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 18 introduced templates; this module combines them with everything students have since learned about classes. Students define a class template's members outside the class and learn why template code must live in headers; write member function templates; use non-type template parameters to build a fixed-capacity ring buffer; specialise templates for one type; combine templates with inheritance (including the `this->` rule for dependent bases); compare static and dynamic polymorphism and recognise CRTP; and use C++20 concepts to state what a template requires, replacing pages of template errors with one clear message.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Define class template members outside the class, and explain why template code belongs in headers.
- Write member function templates, in ordinary classes and in class templates.
- Use non-type template parameters, and explain why the values must be compile-time constants.
- Write full specialisations of function and class templates, and judge when they help.
- Combine templates with inheritance, including using `this->` with a dependent base.
- Compare static and dynamic polymorphism and choose between them; recognise CRTP.
- Define and use C++20 concepts to constrain templates.

Prerequisites

- Part 2 Module 18 (templates).
- Module 22 (headers and linking) and Module 29 (abstract classes).

Materials & Instructor Prep

- 32-templates-meet-oop/examples/ 01 to 07; 07_concepts.cpp needs `-std=c++20`.
- Prepare the 'template definition in a .cpp file' linker error to show live.

Key Vocabulary

Term	Definition
Class template	A recipe for classes, such as <code>Statistics<T></code> ; each type used generates a separate real class.

Term	Definition
Member function template	A method that is itself a template, such as <code>template <typename F> void forEach(F f)</code> .
Non-type template parameter	A compile-time value used as a template argument, such as the <code>N</code> in <code>RingBuffer<double, 8></code> .
Specialisation	A separate definition of a template used instead of the general one for one particular type.
Static polymorphism	Generic behaviour chosen at compile time through templates, needing no common base class.
CRTP	The Curiously Recurring Template Pattern: <code>class X : public Base<X></code> , giving shared base-class code with compile-time dispatch.
Concept (C++20)	A named compile-time test on a type that constrains which types a template accepts.

Lesson Outline & Instructor Talking Points

1. Out-of-class definitions and headers (15 min)

- Run `examples/01_class_template_out_of_class.cpp` and point to the repeated template `<typename T>` and `Statistics<T>::`.
- Move a definition into a separate `.cpp` file and show the 'undefined reference' at link time. Explain: the compiler needs the recipe wherever the template is used.

Instructor note: Connect this to Module 22 explicitly. Students who understood separate compilation will find the rule obvious; those who didn't get a second chance here.

2. Member templates and non-type parameters (20 min)

- Run `examples/02_member_function_template.cpp`: one `addAll()` accepts a vector, a set and an array.
- Run `examples/03_non_type_parameter.cpp`. Show `RingBuffer<double, 4>` and `RingBuffer<int, 3>` are different types, and that a runtime variable can't be used as `N`.

3. Specialisation (10 min)

- Run `examples/04_specialization.cpp`. Show that `Statistics<bool>` has no `mean()` at all - it's a different class with the same name.
- Discuss when a differently named class would be clearer than a specialisation.

4. Templates with inheritance, static vs dynamic (20 min)

- Run `examples/05_template_inheritance.cpp`: one `TypedChannel<T>` template, three classes, one polymorphic loop. Remove `this->` to show the dependent-base error.
- Run `examples/06_static_vs_dynamic.cpp` and fill in the comparison table together, ending with the rule of thumb.

5. Concepts (15 min)

- Run examples/07_concepts.cpp with -std=c++20. Uncomment the Statistics<std::string> line and compare the error with an unconstrained template's error.
- Have students write a concept describing a class from their own track.

Common Student Misconceptions

Misconception	How to Address It
"Template member definitions go in the .cpp file like any other class."	The compiler must see the full definition wherever the template is used, so definitions stay in the header.
"RingBuffer<double, 8> and RingBuffer<double, 16> are the same type."	Every distinct template argument - including values - produces a different type.
"Templates replace virtual functions."	Templates resolve at compile time and can't hold a runtime-mixed collection; virtual functions can. Large programs use both.
"Concepts make templates slower or change what they do."	Concepts only constrain which types are accepted and improve error messages; the generated code is the same.

Exercises & Module Project

Exercise: 32-templates-meet-oop/exercises/exercise1.cpp - a HasId concept and a Repository<T, MaxItems> class template with out-of-class members, duplicate and capacity errors, and a member function template countIf(), used with Student and Book. Project (three tracks): a header-only toolkit with a concept-constrained Statistics<T>, a specialisation for bool, a RollingWindow<T, N> with forEach, a class template derived from an abstract Column/Channel, and a concept describing the track's own classes.

Facilitation guidance: Check that all template code is in headers, that the commented-out error lines are present, and that the static-vs-dynamic comment in main.cpp makes a real argument. Confirm the project builds with -std=c++20 on the student's own machine.

Discussion & Comprehension Check Questions

1. Why must template definitions be visible in the header?

Guidance: Instantiation happens where the template is used; without the full definition there, the compiler can't generate the code and the linker finds nothing.

2. When would you choose virtual functions over templates?

Guidance: When objects of different types must be handled in one runtime collection, or new types must work without recompiling the code that uses them.

3. What do concepts improve?

Guidance: They state requirements up front and produce clear errors at the point of misuse, instead of errors deep inside the template.

Assessment Ideas

- Students write a concept and a constrained function template for a capability in their track.
- Given a design scenario, students argue for static or dynamic polymorphism.

Answer Key Notes

See 32-templates-meet-oo/exercises/solutions/exercise1_solution.cpp (compile with -std=c++20).

Wrap-Up & Bridge to Next Module

Bridge forward: "You now know essentially every OO feature C++ offers. Next module stops asking 'how do I write this?' and asks 'how should I organise this?' - the SOLID principles."

Instructor Tips

- If time is short, protect templates in headers (step 1) and static vs dynamic (step 4).
- CRTP is for recognition only at this stage; don't let it take over the session.